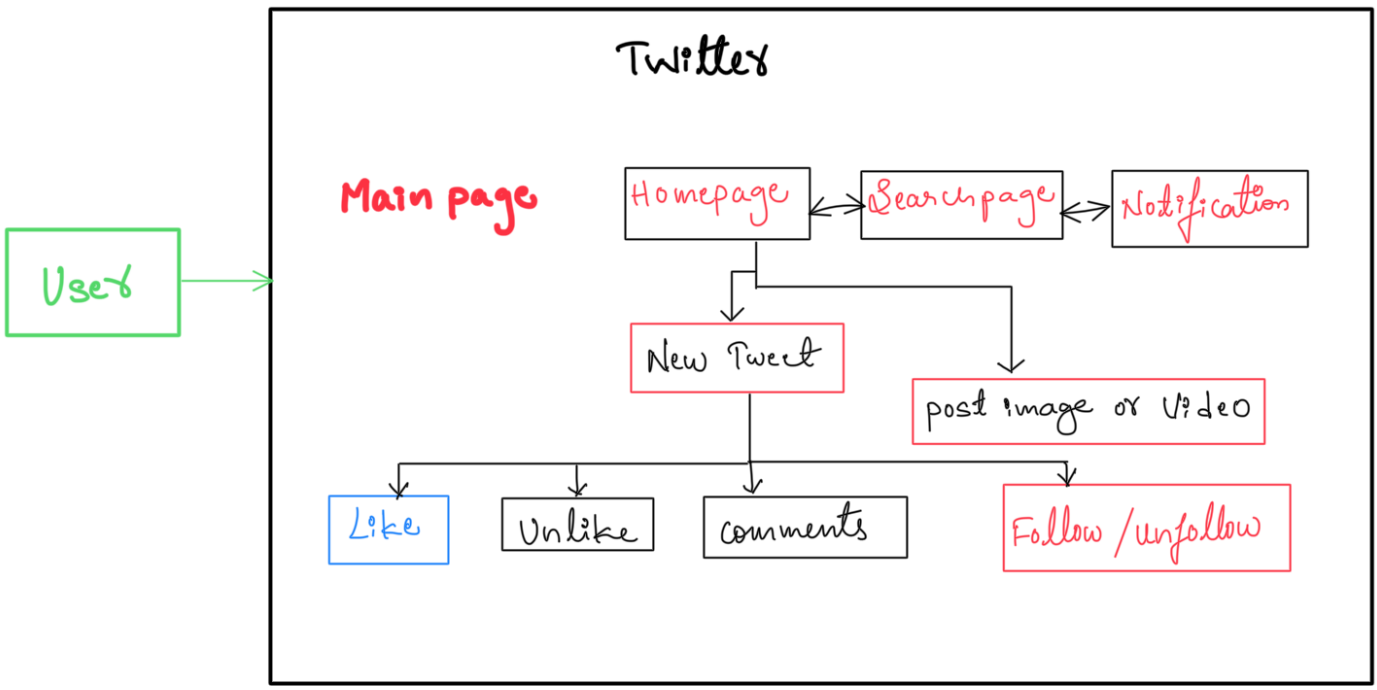
Exploring Twitter System Design - part 2

DAY 106
24/10/25

Use Case Diagram of Twitter



In the above Diagram

- User will click on Twitter page they will get the main page Inside main page, there will be Home page, Search page Notification page.
- Inside Homepage there will be new Tweet page as well as post Image or Videos.
- In new Tweet there we will be like, dislike, comments as well as follows / unfollow Button.
- Guest users will have only access to view any tweet
- Registered use can view and post tweets, can follow & Unfollow other Users
- Registered user will able to create new tweets

Low Level Design for Twitter System Design

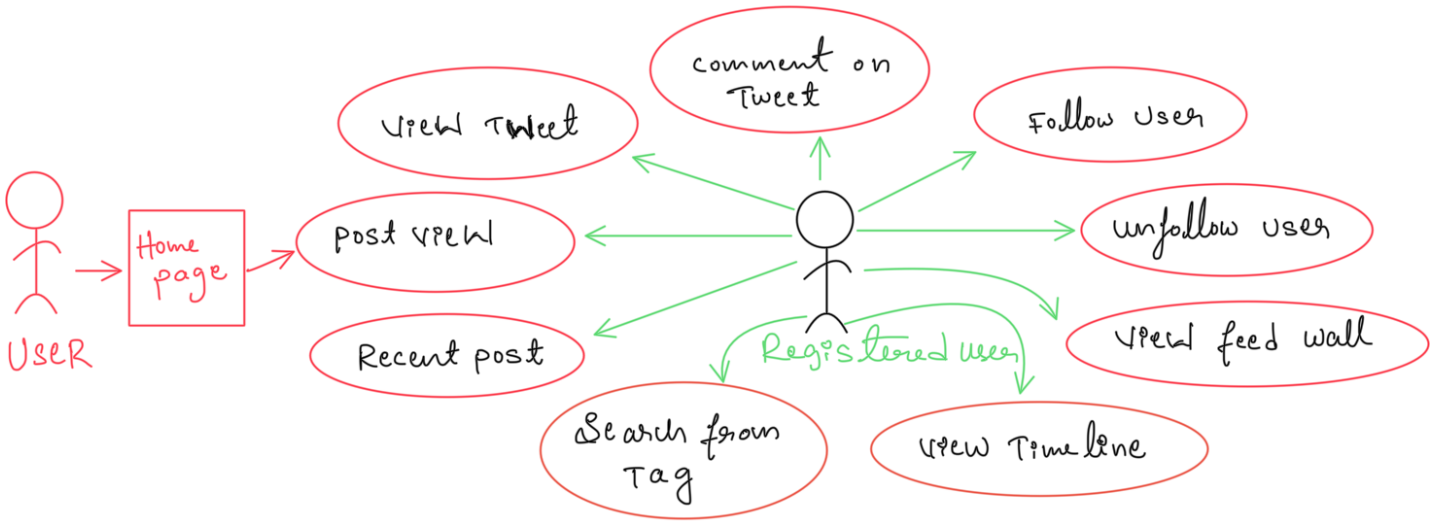
A low-level design of Twitter dives into the details of individual components and functionalities.

Here's a BreakDown of Some key aspects

Data storage (user accounts, Tweets, Follow Relationships, Additional data)

core functionalities (posting a tweet, Timeline generation, Search, Follow or unfollow)

Data storage, Load Balancing, caching, API design, Message Queue



Data storage

- User Accounts : stores User Data like Username, Email, Password (hashed), Profile Picture, Bio, Etc in a Relational DataBase like MySQL
- Tweets : store tweets in a Separate table within the Same DataBase, including tweet, content, author ID, time stamp, hashtags, Mentions, retweets, replies etc
- Follow relationships : Use a Separate table to map followers and followees, allowing efficient retrieval of User feeds
- Additional Data : stores Media assets like images or videos in a dedicated storage System like S3 and reference them in the tweet table.

core functionalities

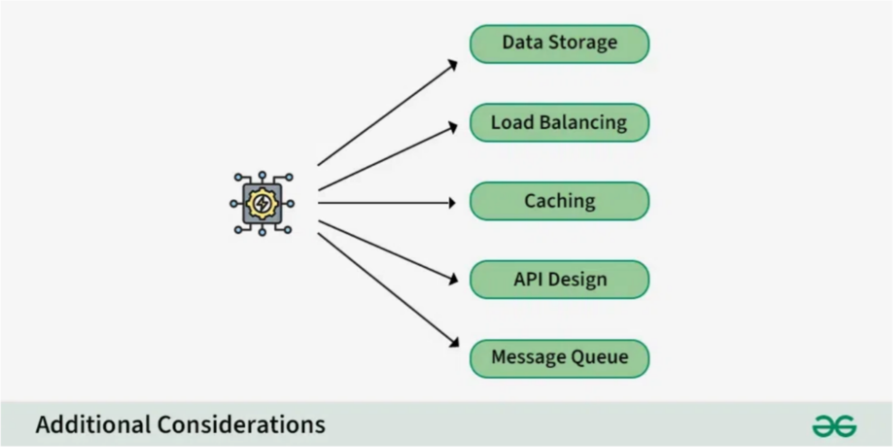
- Posting a tweet
  User Submit content → Validated (length, media, etc) → Stored with hashtags/Mentions → followers and mentions Notified.

- Timeline Generation
  Fetch followed Users / hashtags → retrieve & rank tweets (relevance, recency, Engagement) → Cache in Redis for Speed.

- Search
  process keywords / hashtags → analyze tweets content & metadata → return Ranked Results

- Follow / Un Follow
  Update relationships → adjust timelines → Send Notifications

Additional Considerations



- **Caching:** Use caching mechanisms like Redis to reduce database load for frequently accessed data like user timelines and trending topics.
- **Load balancing:** Distribute workload across multiple servers to handle high traffic and ensure scalability.
- **Database replication:** Ensure data redundancy and fault tolerance with database replication techniques.
- **Messaging queues:** Leverage asynchronous messaging queues for processing tasks like sending notifications or background indexing.
- **API design:** Develop well-defined APIs for internal communication between different components within the system.

Tomorrow I will Explore Twitter's System Design further